

Journal 07: The Risks of Hallucinations in AI Code Generation

Sharie Rhea

Southern New Hampshire University

CS 530: Artificial Intelligence Principles and Applications

Joshua Andrews

March 22, 2026

Computer scientists, software engineers, and programmers now have an incredible tool at their disposal: artificial intelligence (AI). However, developers should be aware of the potential for hallucinations, when plausible but false, incorrect, or misleading outputs are created. This paper first clarifies the types of AI hallucinations regarding code generation and provides their potential underlying causes. The impact of these hallucinations on codebases, products, and the developers themselves is analyzed. Finally, the importance of human oversight and validation when working with AI-generated code is discussed.

A clearer picture of these issues emerges by first categorizing the types of hallucinations that occur in code generation. In the context of code generation, Pavel et al. (2025) have organized hallucinations into four categories. The first two categories encompass errors that are visible and typically easy to detect using automated systems. Syntactical hallucinations occur when generated code does not compile or pass inspection before running, such as mismatched braces or improper indentation (Pavel et al., 2025). Code that compiles but fails or causes an exception during runtime is classified under runtime execution hallucinations (Pavel et al., 2025). The latter two categories house more subtle issues that arise when using AI-generated code. Functional correctness hallucinations occur when the generated code compiles and executes without issues, but does not actually accomplish the desired task (Pavel et al., 2025). This type of error is also often referred to as a “logical bug.” Finally, code quality hallucinations manifest as code that runs and achieves the desired goal, but is either non-optimal or insecure. Mishandling of resources, security vulnerabilities, unnecessarily complicated logic, and code that is difficult to maintain or “smells” all fall under this category (Pavel et al. 2025; Virvou et al., 2025).

These classifications highlight the need to examine the mechanisms that give rise to hallucinations in the first place. Hallucinations are an inherent result of the nature of LLMs. AI models do not possess any knowledge or understanding of the code that they generate, outputs are simply a set of tokens statistically likely to follow each other and the input prompt (Pavel et al., 2025). In this sense it is not possible to completely eliminate hallucinations. However, this insight allows for their reduction by exploiting the structure of the LLM. For example, the likelihood of hallucinations may be reduced by providing enough context in the prompt. AI often creates nonexistent libraries or application programmer interfaces (APIs), so providing a list of endpoints can help mitigate this issue (Faker & Toker, 2026). Additionally, incomplete or incorrect training data can lead to an increased rate of hallucinations (Pavel et al., 2025).

As a result, hallucinations can have far-reaching effects on both technical systems and the developers who rely on them. In codebases, hallucinations greatly increase the risk of bloated, poorly structured, or difficult to interpret code (Pavel et al., 2025). This code “smell” leads to a lack of transparency that compounds as developers struggle to modify and maintain the code, often resorting to bad practices or further use of AI. When AI-generated code makes it to production without proper review it is likely to introduce subtle security or optimization flaws (Faker & Toker, 2026). An even larger concern arises when vulnerabilities are introduced into safety-critical systems, such as those in the health or automotive industries (Pavel et al., 2025). For developers themselves, hallucinations may inadvertently reinforce preheld misconceptions or biases (Virvou et al., 2025). It is possible that AI code generation will lead to a dangerous echo chamber in which a developer holds a misconception that is then reinforced by the “flattering” nature of the LLM and the output is blindly accepted due to authority bias.

Clearly, something must be done to mitigate the risks of utilizing AI-generated code. Since eliminating hallucinations entirely is not feasible, one solution is to enforce robust human oversight and validation. Faker & Toker (2026) propose the CODE framework, consisting of critical assumption checks, objective testing, disciplined code analysis, and epistemological awareness. The first step is designed to check critical assumptions within the code, such as the existence of a library. Next, objective testing validates that the desired task is accomplished, eliminating functional hallucinations. Static code analysis tools can aid in reviewing code for best practices and help to reduce code “smell.” Finally, simple awareness of confirmation and authority bias mitigates the potential for echo chambers.

In conclusion, while AI has become a powerful tool for computer scientists, software engineers, and programmers, it introduces the persistent challenge of hallucinations. Ranging from easily detectable errors to subtle issues in functionality and quality, hallucinations stem from the probabilistic nature of large language models and cannot be fully eliminated. As demonstrated, they affect code correctness, maintainability, security, and even developer judgment. Therefore, careful human oversight, thorough validation, and increased awareness of potential biases are essential to ensure that AI-generated code is used responsibly and effectively.

References

- Faker, O., & Toker, S. (2026). Informed Skepticism: Navigating The Code Hallucination Dilemma. *2026 5th International Informatics and Software Engineering Conference (IISEC)*, Page 634-Page 639. <https://doi.org/10.1109/iisec69317.2026.11418436>
- Pavel, M., Petrovic, N., Mazur, L., Zolfaghari, V., Pan, F., & Knoll, A. (2025). Hallucination in LLM-Based Code Generation: An Automotive Case Study. *2025 3rd International Conference on Foundation and Large Language Models (FLLM)*, 101–106. <https://doi.org/10.1109/fllm67465.2025.11391125>
- Virvou, M., Tsihrintzis, G. A., & Tsichrintzi, E.-A. (2025). Hallucinations in Generative AI: Epistemic Risks for Learners in Educational Applications. *2025 16th International Conference on Information, Intelligence, Systems & Applications (IISA)*, 1–8. <https://doi.org/10.1109/iisa66859.2025.11311276>